Qualcomm SDET Interview - Preparation Summary

Snapdragon Digital Chassis | ADAS / AD Test Engineering

Sumit Debnath - Senior SDET (17+ yrs) - Interview Day: June 17, 2026

Consolidated summary of 11 source documents: the master strategy guide plus question banks on Core Java, Java Coding, Data Structures, Operating Systems, Unix/Linux, GitHub Actions CI/CD, Agentic AI, and ADAS/automotive scenarios.

1. INTERVIEW SCHEDULE & STRATEGY (5 rounds)

Slot / Interviewer	Type	Focus
10:00 Sophie Zhu	Intro / HR / Behavioral	Career story, team fit, why Qualcomm
11:00 Aravind V.	DEEP TECHNICAL (hard)	Automation arch, Python/C++, Linux, ADAS
12:00 Rohit Nair	DEEP TECHNICAL (hard)	HIL/SIL, sensors, OS internals, debugging
14:00 Toan Tran	Quick check / panel	Short - product/team alignment, be concise
15:00 Suishmitaa S.	Behavioral / Process	Leadership, cross-geo teams, test strategy

Key tip: Aravind & Rohit are the hardest - save sharpest technical stories for them. Eat a proper lunch in the 12:45-14:00 break.

2. POSITIONING & KEY DIFFERENTIATORS

Pitch: Senior SDET, 17 yrs in enterprise test automation (Java/Python - Playwright, Selenium, REST Assured) embedded in Jenkins CI/CD. Led a Playwright-Java migration at ServiceNow (40% less manual regression) and built integration validation for distributed data systems (Trino, Zero-Copy Connector). Prior QA ownership of financial compliance products on AWS at Intuit.

Mapping Qualcomm's needs to existing experience

Frameworks: Playwright-Java / Python / REST Assured -> layered framework design transfers directly.

Linux/QNX/Android: Jenkins+Docker, shell scripting, AWS Lambda. Acknowledge QNX gap; show fast-ramp pattern.

HIL/SIL: Pre-prod compliance staging at Intuit parallels SIL; understands the construct.

Distributed teams: Led cross-geo SDETs (San Diego/India/Europe); mentored juniors; standardized automation.

RCA: ZCC/Trino/IntegrationHub deep debugging = strongest parallel to embedded system RCA.

Weakness framing: Embedded/ADAS is the active ramp area - studying ISO 26262, AUTOSAR, sensor-fusion test patterns.

3. BEHAVIORAL - STAR HIGHLIGHTS (Sophie & Suishmitaa)

Framework from scratch: Migrated brittle serial Selenium -> Playwright-Java with POM + DI, parallel + cross-browser, Jenkins per-PR. Result: 40% less manual regression, adopted org-wide.

Leading cross-geo: Standardized conventions, onboarding docs, weekly automation-health syncs, mentored 3 juniors. Ramp time 4-6 wks -> under 2 wks.

Changing requirements: Coverage-tier model (T1 smoke / T2 feature regression / T3 exploratory) + weekly automation-debt review. Mid-cycle coverage gaps down ~60%.

Pushing back on schedule: Risk-based prioritization matrix - 70% of tests covered 95% of business risk; shipped on time with explicit deferred-risk sign-off.

Mentoring: Teach the 'why' first, pair on real bugs, code review as teaching, graduated challenges.

4. DEEP TECHNICAL - AUTOMATION, PYTHON & SYSTEM TEST (Aravind)

Layered ADAS automation framework (e.g. AEB)

L1 test-data (scenarios as YAML/JSON) -> L2 environment abstraction (SIL = simulated feeds e.g. CARLA; HIL = real ECU via CAN) -> L3 execution (pytest scenario injection) -> L4 validation (actual vs ISO 26262 safety threshold) -> L5 reporting (Allure/XML, regression trend). Mirrors the ServiceNow Playwright layering.

SIL vs HIL

SIL: Software on host PC, simulated sensors (CARLA/Simulink/mocks). Fast, cheap, lower fidelity - for algorithm validation & CI regression.

HIL: Real ECU on a test rig, signals via CAN/Ethernet/MIPI. Slower, costly, high fidelity - for timing/real-time, pre-release safety gate. Tools: dSPACE, NI VeriStand, Vector CANoe.

Other key answers

Camera on Linux: V4L2 unit tests -> capture/FPS/dropped-frame checks -> valgrind stress -> error injection. OpenCV/v4l2 + timestamp jitter logging.

REST/telemetry test: pytest + requests: status, schema, type/range (0-250 km/h), pagination tokens - same as ZCC pagination validation.

Flaky tests: Root-cause first (timing/env/logic), logged retries w/ backoff, clean ECU reset per test, timestamp everything, quarantine & track flakiness rate.

5. DEEP TECHNICAL - OS / LINUX INTERNALS & SENSORS (Rohit)

Intermittent embedded fail: Reproduce 100x, add CLOCK_MONOTONIC timing, check /proc (meminfo, schedstat, interrupts), dmesg, strace, diff kernel/driver/compiler vs dev.

Race condition: Outcome depends on concurrent timing (e.g. capture vs detection on shared frame buffer). Test via stress, ThreadSanitizer (-fsanitize=thread), fault injection, lock review.

Process lifecycle: Created->Running->Waiting->Zombie->Dead. Reap children (subprocess timeout/wait), clean shm/fds/semaphores, isolate per container/namespace.

CAN bus: Multi-master, message-ID arbitration. Test functional (send signal, verify ECU), timing/jitter, error-frame -> safe state, validate all DBC signals (python-can).

LiDAR test plan: Interface/protocol -> data integrity (point count, no NaN) -> timing/latency -> detection accuracy (+-5cm) -> edge cases (rain/fog) -> 8h stress -> failure injection -> safe degraded mode.

Sensor fusion: Fuse camera+LiDAR+radar+IMU. Challenges: temporal alignment (diff Hz), coordinate transforms, occlusion, sensor disagreement (choose safe), <100ms end-to-end latency.

6. DOMAIN REFERENCE - ADAS, ISO 26262 & AUTOSAR

ADAS features: AEB (auto braking), LKA (lane keep), ACC (adaptive cruise), BSM (blind spot), FCW (collision warning). Know activation distances, edge cases, latency budgets.

ISO 26262: Functional-safety lifecycle. ASIL A(low)->D(highest, e.g. braking/steering). HARA assigns ASIL; safety goals cascade to SW reqs. Test coverage = safety evidence; freedom from interference.

AUTOSAR: Standard ECU SW architecture. Classic = real-time OS for safety ECUs; Adaptive = Linux/POSIX for high-compute ADAS (Qualcomm's space). BSW, RTE, SWC testing.

ADAS scenarios (extra): OTA updates: safe rollback + integrity/compatibility tests, simulate poor network. Real-time scheduling: priority-based preemption (QNX), watchdogs for missed deadlines. Security: threat analysis, encryption, pen-test of BT/cloud APIs.

7. TECHNICAL QUESTION BANKS (study coverage at a glance)

Core Java (Top 50) Primitives & memory, ==/equals, String pool & immutability, OOP/SOLID, abstract vs interface, exceptions & try-with-resources, Collections, HashMap internals, ConcurrentHashMap, Java 8 streams/Optional/Collectors.

Java SDET (Top 50) Core Java + design patterns (POM, Builder), generics, concurrency (ThreadLocal, ExecutorService, Atomic), testing frameworks (TestNG, Mockito, retry, DataProvider), Playwright/REST Assured/Jenkins.

Java Coding (20 problems) Arrays/Strings (Two Sum, sliding window), linked lists (reverse, cycle), trees, concurrency. Full Java solutions with time/space complexity - expect live coding.

Data Structures (50 Qs) Arrays, two-pointer/Kadane, linked lists, stacks/queues/heaps, trees & BST (traversal, LCA, serialize, AVL, tries, segment tree), graphs (BFS/DFS/Dijkstra/topo/Union-Find), hashing, LRU cache, sorting/QuickSelect, DP (knapsack, LCS, memo vs tab).

Operating Systems (101 + notes) Process/thread states, scheduling, IPC, paging & virtual memory, deadlock (detection/avoidance/recovery), synchronization, fragmentation, file systems, logical vs physical address/MMU, Belady's anomaly, real-time. QNX/embedded focus.

Unix / Linux (Top 25) Paths, find vs locate, file ops, permissions, process mgmt (ps/top/htop), grep/sed/awk, pipes & redirection, /proc /sys /dev for hardware registers, networking & log analysis for CI/CD and ADAS targets.

GitHub Actions CI/CD (Top 25) Workflow YAML (on/jobs/steps/runs-on/needs), GH Actions vs Jenkins, matrix builds, reusable workflows, secrets, self-hosted runners, caching, artifacts, gating. Jenkins experience maps 1:1.

Agentic AI (Top 50) AI agents vs LLM calls, ReAct loop, tool calling, memory, guardrails, multi-agent; RAG (embeddings, vector DBs, hybrid search, reranking, GraphRAG, eval); AI test-gen & self-healing, RCA; LLMOps/LangSmith, prompt injection; LangChain/LlamaIndex/LangGraph/CrewAI/AutoGen/MCP.

8. DAY-OF EXECUTION PLAN

Night before: Review master guide June 16 evening - do not cram morning of. Memorize STAR stories as structure, not scripts.

Energy: Sophie 10am strong first impression -> Aravind/Rohit bring best depth while fresh -> Toan warm & concise -> Suishmitaa end strong on leadership.

Breaks: 15-min resets after each morning round; 12:45-14:00 = lunch + decompress; rest before Suishmitaa.

Close every round: 'Is there anything about my background I can clarify or elaborate on?'

Smart questions to ask

Technical: Current SIL vs HIL split & biggest test pain (coverage/stability/speed)? SIL->HIL gate automated or manual? Which simulation tools for scenario generation?

Team/Product: How are San Diego & India teams structured? 90-day onboarding ramp? Tooling autonomy vs standard toolchain? Which ADAS features are prioritized next? Do SDETs join early design reviews?

Source files summarized: Qualcomm_SDET_Interview_Prep_Sumit, Core_Java_Top50, Java_SDET_Top50, Java_Coding_Top20, DataStructures_Java_Interview_Guide, Operating Systems Notes, OS_Interview_Questions, Unix_Commands_Top25, GitHub_Actions_CICD_Top25, Agentic_AI_Top50, SomeMoreQuestion.